



SIMONE DE BEAUVOIR
ÉCOLE FRANÇAISE
FONDATION NOUR EL MAAREF - 1992

1ère
NSI

Chapitre 2 : Algorithmique et programmation

Leçon 3: Documentation et Tests d'un Programme en Python

Thème : Langages et programmation

Enseignant : Houssem Lahiani

Introduction

Imaginez construire une maison sans plan ni inspection - une tâche presque impossible et certainement imprudente. De la même manière, dans le domaine du développement logiciel, la documentation et les tests sont les piliers sur lesquels repose la robustesse, la fiabilité et la maintenance des programmes.

Rôle Crucial de la Documentation

La documentation est la clé qui ouvre les portes de la compréhension. C'est comme le manuel d'utilisation d'un nouvel appareil. Imaginez si chaque fonction ou chaque ligne de code était une énigme sans explication. La documentation, en décrivant le but, le fonctionnement et les exigences d'une partie de code, permet à toute personne - qu'il s'agisse du développeur initial, d'un collègue ou d'un futur mainteneur - de comprendre rapidement et efficacement ce qu'une portion spécifique de code fait et comment l'utiliser correctement.

Importance Cruciale des Tests

Les tests sont les gardiens de la qualité logicielle. Imaginons maintenant que la documentation soit le plan de la maison. Les tests, eux, sont les inspections minutieuses qui s'assurent que chaque mur est solide, chaque fil électrique est correctement connecté, et que rien n'est laissé au hasard. Les tests garantissent que le programme fonctionne comme prévu, même dans les conditions les plus extrêmes. Ils identifient les faiblesses potentielles, évitant ainsi les erreurs coûteuses et améliorant la confiance dans la stabilité du logiciel.

Ensemble Indissociable

La documentation et les tests ne sont pas simplement deux aspects distincts du développement logiciel, mais plutôt un ensemble indissociable. Une documentation claire permet aux tests d'être bien conçus, et des tests complets génèrent des informations précieuses pour la documentation. Ensemble, ils forment un duo puissant qui assure la qualité, la maintenance et l'évolutivité des logiciels.

1. Spécification et Prototypage

La spécification précise des fonctions et la création de prototypes sont cruciales en développement logiciel. Une spécification claire détaille le comportement attendu d'une fonction, facilitant la compréhension et la collaboration entre les développeurs. Les prototypes, bien que non obligatoires en Python, permettent d'annoter les types d'arguments et de retour, renforçant la documentation et facilitant la détection précoce d'erreurs. Le prototypage de fonction en Python se fait en utilisant des docstrings. Une docstring est une chaîne de caractères placée en début de fonction qui décrit le rôle de la fonction, le type des paramètres et le type de la valeur de retour.

Par exemple, considérons la fonction d'addition suivante :

```
def addition(a: int, b: int) -> int:
    """
    Cette fonction prend deux entiers en argument et retourne
    leur somme.

    Args:
        a (int): Le premier entier.
        b (int): Le deuxième entier.

    Returns:
        int: La somme de a et b.
    """
    return a + b
```

Ici, la spécification détaillée et le prototype annoté clarifient l'objectif de la fonction, guidant les développeurs lors de son utilisation et contribuant à un code plus fiable.

Il faut taper `help(addition)` et voir le résultat de la documentation

2. Préconditions sur les Arguments

La définition de préconditions sur les arguments d'une fonction est une pratique essentielle pour garantir le bon fonctionnement du programme. Les préconditions spécifient les conditions que les arguments doivent remplir afin que la fonction s'exécute correctement. En Python, bien que le langage ne contraigne pas formellement les types d'arguments, il est courant d'énoncer explicitement ces préconditions dans le code, comme illustré dans l'exemple ci-dessous :

```
def somme_liste(liste):
    """
    Cette fonction prend une liste d'entiers en argument et
    retourne leur somme.

    Args:
        liste (list): Une liste d'entiers.

    Returns:
        int: La somme des entiers dans la liste.

    Precondition:
```

```
La liste doit être une liste d'entiers.  
""  
  
if all(isinstance(element, int) for element in liste):  
    return sum(liste)  
  
else:  
    raise ValueError("La liste doit être une liste  
d'entiers.")
```

Dans cet exemple, la fonction `somme_liste` exige que l'argument `liste` soit une liste d'entiers. Avant de calculer la somme, la fonction vérifie si tous les éléments de la liste sont de type entier. Si la précondition n'est pas satisfaite, elle déclenche une exception de valeur, indiquant que la liste doit être une liste d'entiers. Cette approche renforce la robustesse du programme en imposant des contraintes sur les données en amont, évitant ainsi des erreurs potentielles lors de l'exécution de la fonction.

La précondition dans la fonction `somme_liste` est là pour garantir que la liste `liste` est bien une liste d'entiers. Si la liste n'est pas une liste d'entiers, la fonction `somme_liste` ne pourra pas fonctionner correctement.

Dans l'exemple que vous avez fourni, la précondition est vérifiée par l'instruction `all(isinstance(element, int) for element in liste)`. Cette instruction renvoie `True` si tous les éléments de la liste `liste` sont des entiers, et `False` sinon.

Si la précondition n'est pas remplie, la fonction `somme_liste` lèvera une exception `ValueError`. Cette exception indiquera au programmeur que la liste n'est pas une liste d'entiers.

Dans l'exemple que vous avez fourni, la fonction `somme_liste` est appelée avec la liste `[1, 2]`. Cette liste est une liste d'entiers, donc la précondition est remplie. La fonction `somme_liste` renvoie donc la somme des éléments de la liste, soit 3.

Voici un exemple de ce qui se passe si la précondition n'est pas remplie :

```
somme_liste(["a", "b"])
```

Dans cet exemple, la fonction `somme_liste` est appelée avec la liste `["a", "b"]`. Cette liste n'est pas une liste d'entiers, donc la précondition n'est pas remplie. La fonction `somme_liste` lèvera donc une exception `ValueError`.

```
Traceback (most recent call last):  
  File "main.py", line 1, in <module>  
    somme_liste(["a", "b"])  
  File "main.py", line 7, in somme_liste  
    raise ValueError("La liste doit être une liste d'entiers.")  
ValueError: La liste doit être une liste d'entiers.
```

La précondition dans la fonction `somme_liste` est donc utile pour garantir que la fonction fonctionne correctement. Elle aide à prévenir les erreurs et à rendre le code plus robuste.

3. Assertions pour Garantir les Préconditions

Utilisation d'assertions pour vérifier les préconditions et assurer la robustesse du programme.

Les assertions en Python sont un moyen efficace de vérifier les préconditions et d'assurer la robustesse du programme. Elles permettent de valider certaines conditions au sein du code et, si la condition n'est pas satisfaite, déclenchent une exception `AssertionError`. Dans le contexte des préconditions, les assertions peuvent être utilisées pour renforcer les contraintes sur les arguments d'une fonction. Reprenons l'exemple précédent en utilisant des assertions :

```
def somme_liste(liste):  
    """  
    Cette fonction prend une liste d'entiers en argument et retourne leur somme.  
    Args:  
    liste (list): Une liste d'entiers.  
    Returns:  
    int: La somme des entiers dans la liste.  
    Precondition:  
    La liste doit être une liste d'entiers.  
    """  
    assert isinstance(liste, list), "Erreur : l'argument doit être une liste"  
    assert all(isinstance(x, int) for x in liste), "Erreur : la liste doit  
    contenir uniquement des entiers"  
    return sum(liste)
```

Dans cet exemple, les deux assertions sont utilisées pour vérifier les préconditions. La première assertion s'assure que l'argument `liste` est bien une liste, et la deuxième assertion garantit que tous les éléments de la liste sont des entiers. Si l'une de ces conditions n'est pas respectée, une exception `AssertionError` est levée, indiquant clairement la nature de l'erreur. L'utilisation d'assertions renforce la fiabilité du programme en signalant immédiatement les problèmes liés aux préconditions, facilitant ainsi le processus de débogage et améliorant la qualité du code.

4. Postconditions sur les Résultats

La définition des postconditions sur les résultats d'une fonction est une pratique importante pour garantir la fiabilité des données produites. Les postconditions spécifient les garanties que la fonction offre quant à la valeur de retour ou à l'état du programme après son exécution. Dans l'exemple ci-dessous, une postcondition est ajoutée à la fonction `somme_liste` pour spécifier que le résultat retourné est la somme des éléments de la liste fournie en argument :

```
def somme_liste(liste):  
    """  
    Cette fonction prend une liste d'entiers en argument et retourne leur  
    somme.  
    Args:  
    liste (list): Une liste d'entiers.  
    Returns:  
    int: La somme des entiers dans la liste.  
    Precondition:  
    La liste doit être une liste d'entiers.  
    Postcondition:  
    Le résultat est la somme des éléments de la liste.  
    """  
    assert isinstance(liste, list), "Erreur : l'argument doit être une liste"  
    assert all(isinstance(x, int) for x in liste), "Erreur : la liste doit  
    contenir uniquement des entiers"  
    result = sum(liste)  
    assert result == sum(liste), "Erreur : la postcondition est violée"  
    return result
```

Dans cet exemple, la postcondition garantit que le résultat retourné par la fonction `somme_liste` est effectivement la somme des éléments de la liste en entrée. L'ajout de postconditions est essentiel pour documenter clairement le comportement attendu de la fonction, ce qui facilite son utilisation correcte et contribue à la confiance dans les résultats produits.

En adoptant des préconditions et des postconditions, le programmeur peut renforcer la spécification de la fonction, fournissant ainsi des indications claires aux utilisateurs et aux autres développeurs sur les conditions requises avant l'appel de la fonction et sur les garanties offertes par les résultats après son exécution.

5. Mise au point de programmes : tester son programme

Parfois, un programmeur peut décrire et documenter adéquatement une fonction même si cette dernière n'est pas encore correcte, car des erreurs peuvent survenir lors de la rédaction du code. Afin d'identifier ces erreurs potentielles, il est essentiel de tester le programme. Plutôt que de procéder à des tests individuels, il est plus judicieux de créer des ensembles complets de tests. Cette approche offre une vue d'ensemble des performances du programme dans divers scénarios. Pour faciliter ce processus, les jeux de tests peuvent être intégrés directement dans le programme à l'aide d'instructions `assert`, fournissant ainsi une méthode pratique pour valider les conditions spécifiées. En résumé, la spécification, la documentation, les tests, et l'utilisation d'asserts contribuent collectivement à la création de programmes plus fiables et robustes, permettant aux programmeurs de détecter et de corriger les erreurs plus efficacement. L'utilisation de jeux de tests est une pratique fondamentale dans le processus de mise au point de programmes. Les jeux de tests consistent en un ensemble de cas de test soigneusement conçus pour évaluer le comportement d'une fonction ou d'un programme dans différentes situations. Cette approche contribue à garantir la qualité et la fiabilité du code en identifiant les erreurs potentielles et en vérifiant que le programme se comporte comme prévu dans une

variété de scénarios. L'importance de la qualité et du nombre de tests est cruciale pour s'assurer de la robustesse d'un programme. Un jeu de tests exhaustif doit couvrir différents cas, y compris les situations normales et les cas limites, afin de détecter des problèmes potentiels dans le code. La qualité des tests réside dans leur capacité à révéler des comportements indésirables ou des erreurs de logique.

➔ Les jeux de tests sont souvent conçus pour évaluer le comportement général d'une fonction, tandis que les assertions intégrées dans le code peuvent aider à détecter rapidement des erreurs pendant le développement.

Voici un exemple simple de fonction en Python avec des préconditions et des postconditions, ainsi qu'un jeu de tests associé :

```
def division_secur(a, b):  
    """  
    Cette fonction prend deux nombres en argument et retourne leur division  
    sécurisée.  
    Args:  
    a (float): Le numérateur.  
    b (float): Le dénominateur (ne doit pas être égal à zéro).  
    Returns:  
    float: Le résultat de la division a / b.  
    Precondition:  
    - b ne doit pas être égal à zéro.  
    Postcondition:  
    - Le résultat de la division est renvoyé.  
    """  
    # Précondition : Vérifier que le dénominateur n'est pas égal à zéro  
    assert b != 0, "Erreur : Le dénominateur ne doit pas être égal à zéro."  
    result = a / b  
    # Postcondition : Vérifier que le résultat est effectivement la division  
    de a par b  
    assert result == a / b, "Erreur : La postcondition est violée."  
    return result
```

Et voici un jeu de tests pour cette fonction :

```
# Jeu de tests pour la fonction division_secur  
assert division_secur(10, 2) == 5, "Erreur : le résultat attendu  
est 5"  
assert division_secur(0, 5) == 0, "Erreur : le résultat attendu  
est 0"  
assert division_secur(-8, 4) == -2, "Erreur : le résultat attendu  
est -2"
```

Exercice :

Dans cet exercice, nous allons explorer l'importance des jeux de tests dans le processus de développement logiciel en utilisant un exemple de fonction mal développée. La fonction `appartient_a` prétend tester si un nombre donné appartient à une liste d'entiers. Cependant, il y a des erreurs dans le code qui peuvent passer inaperçues sans des tests appropriés.

La première chose à noter est que la boucle `for` commence à l'indice 1, ignorant ainsi le premier élément de la liste. Cela peut conduire à des résultats incorrects lors de la recherche du nombre

dans la liste. Pour mieux comprendre ces erreurs, nous avons créé des jeux de tests qui devraient réussir si la fonction est correcte, mais qui déclencheront des erreurs si des problèmes existent.

Instructions:

Examinez attentivement la fonction `appartient_a` et identifiez les préconditions et la postcondition définies.

Observez comment la fonction est censée fonctionner et identifiez les erreurs potentielles.

Exécutez les jeux de tests fournis en dessous de la fonction.

Analysez les erreurs générées par les assertions dans les jeux de tests et essayez de comprendre pourquoi elles se produisent.

Proposez des corrections pour résoudre les problèmes identifiés dans la fonction.

Réexécutez les jeux de tests avec la version corrigée pour vérifier si les erreurs ont été résolues.

Cet exercice vise à illustrer comment les jeux de tests peuvent être utilisés pour détecter des erreurs dans le code et renforcer l'importance de tests complets et rigoureux dans le processus de développement logiciel.

```
def appartient_a(nombre, liste):  
    """  
    Cette fonction teste si le nombre en premier paramètre appartient à la  
    liste en deuxième paramètre.  
  
    Préconditions:  
    - nombre doit être de type int.  
    - liste doit être une liste d'entiers.  
  
    Postcondition:  
    - Le résultat doit être de type booléen.  
    """  
    assert type(nombre) == int, "Erreur : nombre doit être de type entier."  
    assert type(liste) == list, "Erreur : liste doit être de type list."  
    assert all(isinstance(x, int) for x in liste), "Erreur : les éléments de  
    liste doivent être de type entier."  
  
    exist = False  
    for i in range(len(liste)):  
        if nombre == liste[i]:  
            exist = True  
            break  
  
    assert type(exist) == bool, "Erreur : la fonction doit retourner une  
    valeur booléenne."  
    return exist  
  
# Jeux de test  
assert not appartient_a(5, [2, 3, 4]), "Erreur : 5 ne fait pas  
partie de la liste."  
assert not appartient_a(10, [2, 3, 4]), "Erreur : 10 ne fait pas  
partie de la liste."
```



```
assert appartient_a(2, [2, 3, 4]), "Erreur : 2 fait partie de la liste."
```

- Comment repérer les erreurs ? → en commençant par un test qui ne réussit pas en suivant l'état des variables

- Mentalement ou sur papier

- Ou à l'aide d'outils numériques :

- Première option : afficher les valeurs de certaines variables

- Deuxième option : exécuter le programme ligne par ligne (disponible dans la plupart des IDE ou sur Python Tutor)

- Troisième option : faire usage d'un débogueur intégré à un IDE (ou debugger)

L'exécution ligne par ligne d'un programme, également appelée "débogage pas à pas", est une fonctionnalité courante dans de nombreux environnements de développement intégrés (IDE), y compris Jupyter Notebook. Cette approche permet aux développeurs d'examiner de près l'exécution du programme à chaque étape, d'inspecter les valeurs des variables et de détecter les erreurs plus facilement.

Dans le cas de Jupyter Notebook, vous pouvez utiliser la commande magique `%debug` pour activer le débogueur interactif. Voici comment vous pourriez l'appliquer à votre code :

1. Incluez la commande `%debug` à l'endroit où vous souhaitez déclencher le débogueur. Vous pouvez le mettre dans la cellule juste après l'appel de la fonction, comme ceci :

```
```python
 assert not appartient_a(5, [2, 3, 4]), "Erreur : 5 ne fait pas
partie de la liste."
```
```

2. Exécutez la cellule. Lorsque l'assertion échoue, le débogueur interactif sera activé, vous permettant d'inspecter l'état du programme.

Cela vous permettra d'examiner chaque étape de l'exécution du programme, d'évaluer les expressions, d'inspecter les variables et de comprendre où l'erreur se produit. C'est une méthode puissante pour comprendre le comportement du code et déceler les erreurs.

Remarque : Assurez-vous que Jupyter Notebook est configuré avec IPython, car la commande `%debug` utilise IPython pour fournir une interface interactive de débogage.

6. Intégration des Tests à la Chaîne de Documentation

L'intégration des tests à la chaîne de documentation combine la description du code avec des exemples concrets de son utilisation. Cela crée une documentation vivante qui ne se contente pas de décrire la fonction, mais qui la met également à l'épreuve à travers des jeux de tests.

a. Documentation de la Fonction

La documentation de la fonction doit inclure une explication claire de son but, de ses paramètres, de ses préconditions, et de ses postconditions. Utilisez des commentaires en ligne ou des docstrings pour capturer ces informations.

```
def somme_liste(liste):  
    """  
    Cette fonction prend une liste d'entiers en argument et retourne leur  
    somme.  
  
    Args:  
    liste (list): Une liste d'entiers.  
  
    Returns:  
    int: La somme des entiers dans la liste.  
  
    Precondition:  
    La liste doit être une liste d'entiers.  
  
    Postcondition:  
    Le résultat est la somme des éléments de la liste.  
    """  
    assert isinstance(liste, list), "Erreur : l'argument doit être une liste"  
    assert all(isinstance(x, int) for x in liste), "Erreur : la liste doit  
    contenir uniquement des entiers"  
    result = sum(liste)  
    assert result == sum(liste), "Erreur : la postcondition est violée"  
    return result
```

b. Exemples de Tests dans la Documentation

Intégrez des exemples de tests dans la documentation en utilisant des blocs de code. Cela fournit des exemples concrets de l'utilisation de la fonction et aide les utilisateurs à comprendre comment elle doit être appelée.

```
def somme_liste(liste):  
    """  
    Cette fonction prend une liste d'entiers en argument et retourne leur  
    somme.  
  
    Args:  
    liste (list): Une liste d'entiers.
```

Returns:
int: La somme des entiers dans la liste.

Precondition:
La liste doit être une liste d'entiers.

Postcondition:
Le résultat est la somme des éléments de la liste.

Exemples:

```
>>> somme_liste([1, 2, 3])  
6
```

```
>>> somme_liste([-1, -2, -3])  
-6
```

```
>>> somme_liste([])  
0
```

```
>>> somme_liste([5])  
5  
"""
```

```
assert isinstance(liste, list), "Erreur : l'argument doit être une liste"  
assert all(isinstance(x, int) for x in liste), "Erreur : la liste doit  
contenir uniquement des entiers"  
result = sum(liste)  
assert result == sum(liste), "Erreur : la postcondition est violée"  
return result
```

c. Utilisation de doctest

Pour rendre ces exemples de tests exécutables, vous pouvez utiliser le module `doctest` de Python. Cela permet d'exécuter les exemples de tests directement depuis la documentation.

Syntaxe à respecter avec les trois chevrons (>>>)

```
def somme_liste(liste):  
    """  
    Cette fonction prend une liste d'entiers en argument et retourne leur  
    somme.
```

```
    Args:  
    liste (list): Une liste d'entiers.
```

```
    Returns:  
    int: La somme des entiers dans la liste.
```

```
    Precondition:  
    La liste doit être une liste d'entiers.
```

```
    Postcondition:  
    Le résultat est la somme des éléments de la liste.
```

```
    Exemples:
```

```
>>> somme_liste([1, 2, 3])  
6
```

```
>>> somme_liste([-1, -2, -3])
```

-6

```
>>> somme_liste([])
0

>>> somme_liste([5])
5
"""
assert isinstance(liste, list), "Erreur : l'argument doit être une liste"
assert all(isinstance(x, int) for x in liste), "Erreur : la liste doit
contenir uniquement des entiers"
result = sum(liste)
assert result == sum(liste), "Erreur : la postcondition est violée"
return result
```

Dans la prochaine cellule de votre fichier jupyter taper ces deux lignes de code :

```
import doctest
doctest.testmod()
```

Il est envisageable d'inclure un paramètre facultatif lors de l'utilisation de la fonction testmod pour obtenir des détails supplémentaires sur les tests, même lorsque ces derniers sont réussis.

```
doctest.testmod(verbose = True)
```

En utilisant `doctest`, vous pouvez automatiquement vérifier si les exemples de tests dans la documentation produisent les résultats attendus. Cela garantit que la documentation reste à jour et précise tout au long du développement.